





Testing Spring Boot Applications Demystified

A Hero's Journey Through the Spring Boot Testing Labyrinth

Talk @ Deichmann SE 04.08.2026

Philip Riecks - [PragmaTech GmbH](#) - [@rieckpil](#)

About Philip

- Self-employed developer from Herzogenaurach (HQ of adidas & Puma), Germany (Bavaria) 🍺
- Blogging & content creation with a focus on testing Java and specifically Spring Boot applications 🌿
- Founder of PragmaTech GmbH - Enabling Developers to Frequently Deliver Software with More Confidence 🚀



Participate During the Talk

Go to menti.com and use the code **7495 6357** to **anonymously** submit answers for the quizzes and add your questions during the talk.



Please answer the **first three questions**:

- Who wrote your tests today?
- How would you rate your Spring Boot testing knowledge?
- How confident are you deploying on a Friday afternoon?

Why Test Software?



Being a Developer in 2026 (?)



Uncle Bob Martin ✓
@unclebobmartin



I'm significantly older than you. I started coding in the late 60s. My current strategy is to not read any of the code written by my agents. That's the only way I can take advantage of their productivity. What I do instead is to surround the agents with extreme constraints. Unit tests, gherkin tests, QA procedures, quality metrics, mutation testing, test coverage, and a plethora of others. In the end, I have very high confidence in the code they produce because they've had to run the gauntlet of all of my constraints and tests.

How not to: Agentic Development



Offload Everything to the AI



Frog told the LLM to write tests for all the code it wrote.

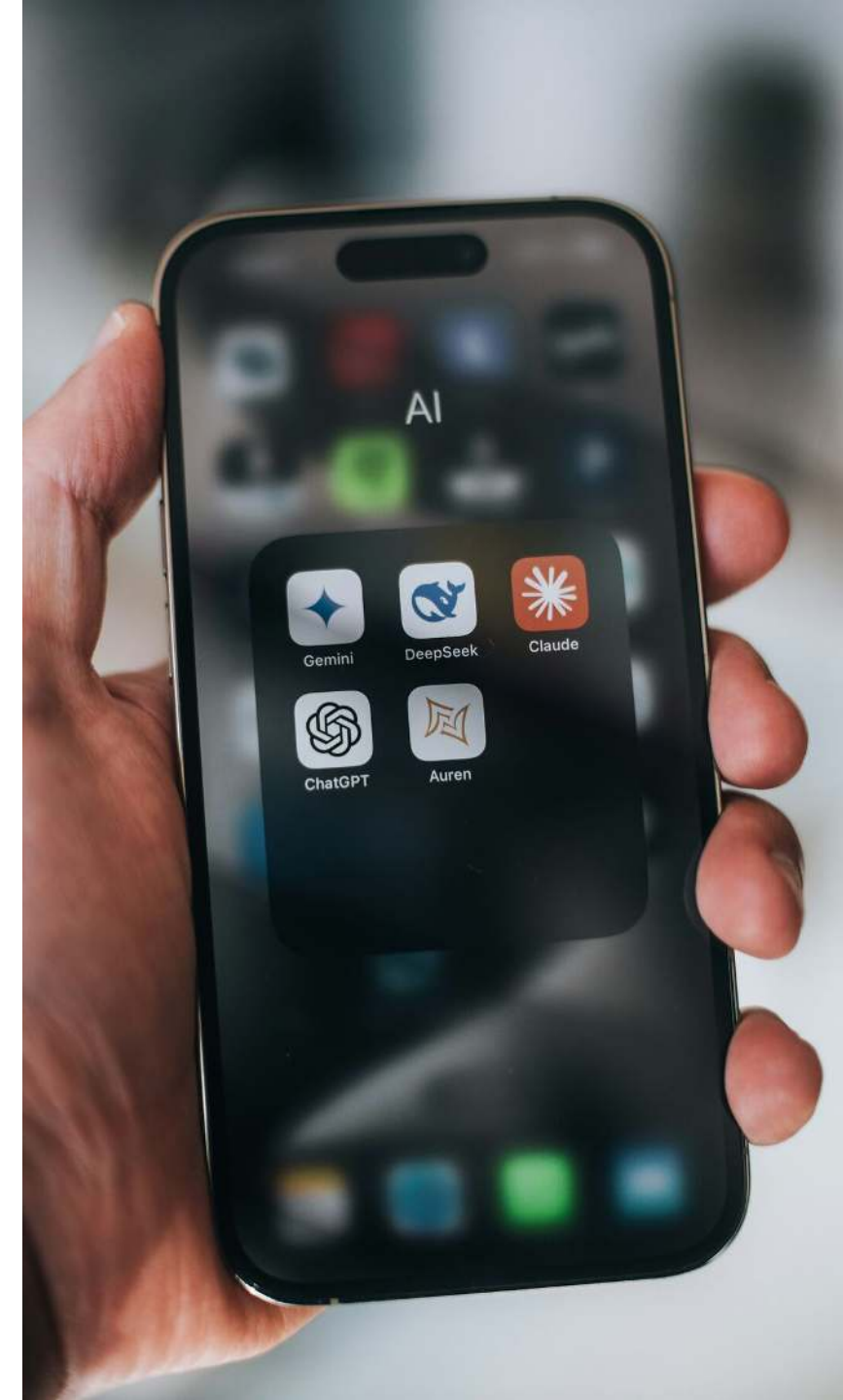
“There,” he said. “Now the model can’t write broken or buggy code.”

“But the model can write broken or buggy tests,” said Toad.

“That is true,” said Frog.

Agentic Development Today

- AI makes software faster to create and harder to trust.
- AI made writing code cheap. Verifying it is the new bottleneck.
- AI adoption correlates with increased throughput and decreased stability.
- AI generates the logic, but you inherit the **liability**. It can write the function; it won't join the post-mortem.

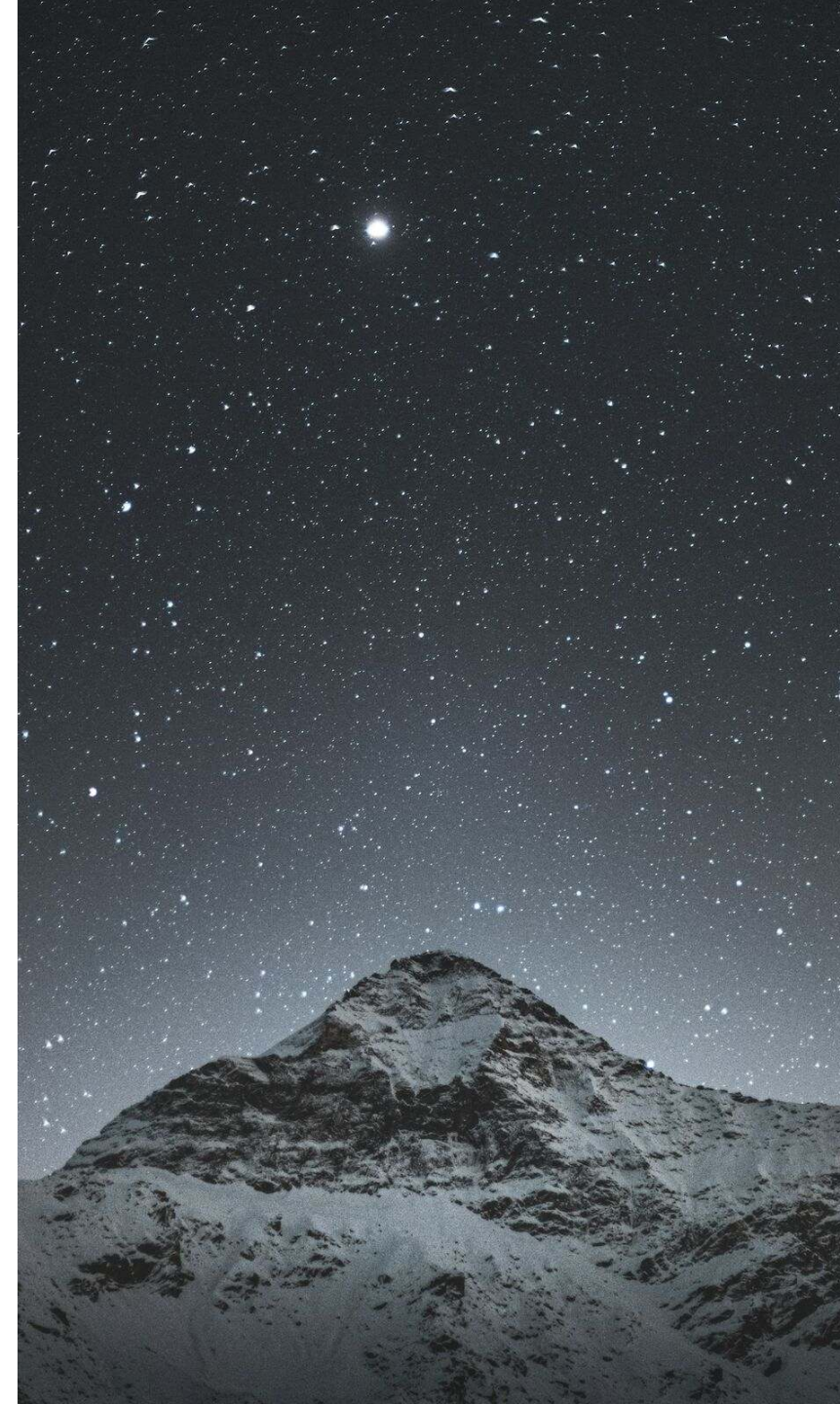


My Overall Northstar for Engineering Excellence

Imagine seeing this pull request on a Friday afternoon:



How confident are you to merge this major Spring Boot upgrade and deploy it to production once the pipeline turns green?



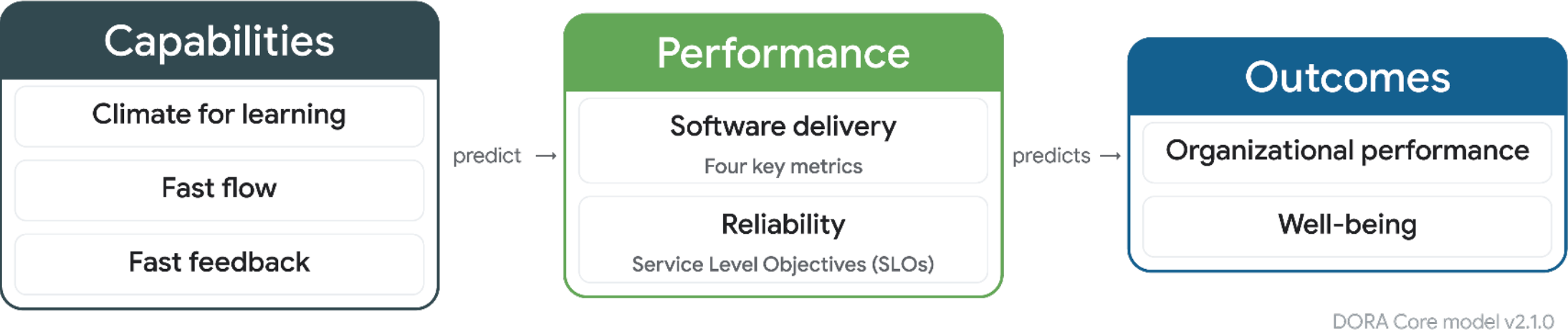
Good tests don't just catch bugs - they give you fast feedback and confident deployments.



DORA (DevOps Research Assessment)

DORA Core Model

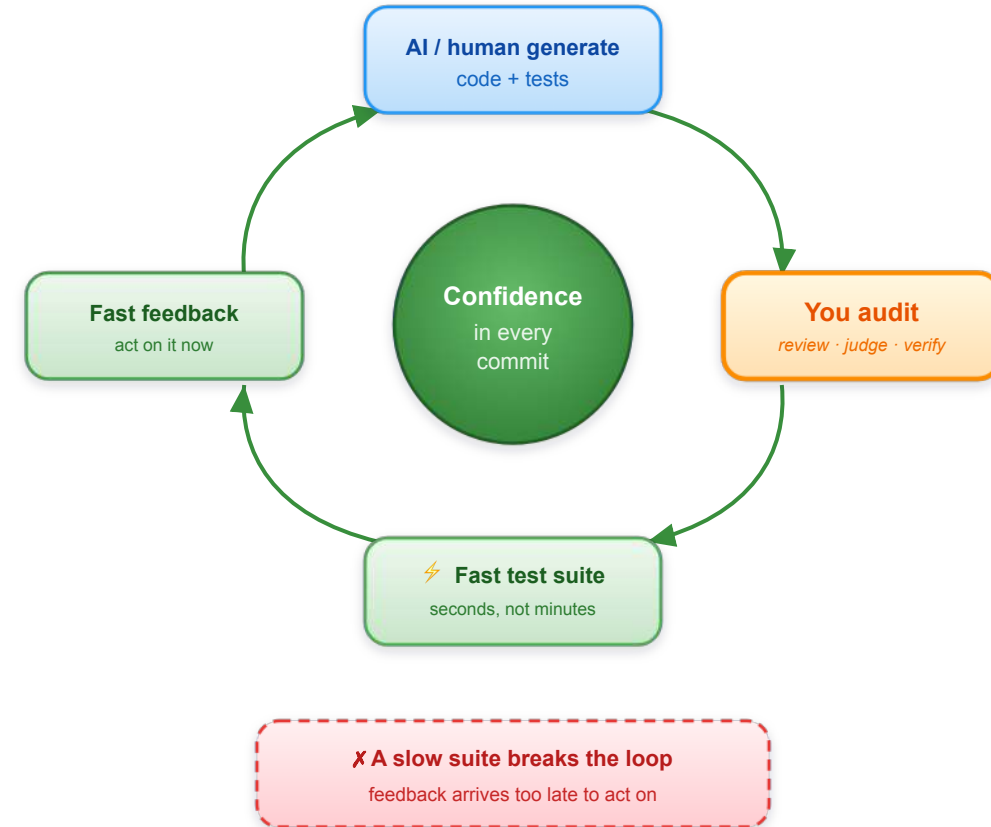
dora.dev/research



From Developer to AI-Code Auditor

- AI writes more code, faster – your job becomes **auditing** it with confidence
- An auditor lives or dies by **fast feedback**: a slow suite breaks the loop
- Fast, reliable tests are the auditor's instrument – the foundation for **confidence in every commit**

The Auditor's Feedback Loop



How do we get there for your **Spring Boot** projects?



The Hero's Journey aka. Our Agenda



Goals For This Talk

- **Provide a clear mental map** for choosing between unit, slice, and integration tests
- Get **better judgment** for AI-written tests
- Collect **practical techniques and best practices** to speed up test suites and validate test quality
- **Build confidence** into your development work to ship fearlessly



Act 1: The Grand Entrance

Testing Spring Boot applications can feel like entering a labyrinth blindfolded:

- Copying test configuration from AI/StackOverflow hoping it works
- The paradox of choice: `@SpringBootTest`, `@WebMvcTest`, `@DataJpaTest`, `@MockBean`, etc.
- Struggling with Spring `ApplicationContext` creation during tests
- Uncertainty about what to test and how to test it effectively



Quest 1

The Unit Testing Guardian

The Swift Gatekeeper - Blocks Those Who Overcomplicate



Our Foundation: Spring Boot Starter Test

- The "Testing Swiss Army Knife"

```
1 <dependency>
2   <groupId>org.springframework.boot</groupId>
3   <artifactId>spring-boot-starter-test</artifactId>
4   <scope>test</scope>
5 </dependency>
```

- Batteries-included for testing by transitively including popular testing libraries
- Out-of-the-box dependency management to ensure compatibility



```
1 ./mvnw dependency:tree
2 [INFO] ...
3 [INFO] +- org.springframework.boot:spring-boot-starter-test:jar:4.0.2:test
4 [INFO] | +- org.springframework.boot:spring-boot-test:jar:4.0.2:test
5 [INFO] | +- org.springframework.boot:spring-boot-test-autoconfigure:jar:4.0.2:test
6 [INFO] | +- com.jayway.jsonpath:json-path:jar:2.10.0:test
7 [INFO] | | \- org.slf4j:slf4j-api:jar:2.0.17:compile
8 [INFO] | +- jakarta.xml.bind:jakarta.xml.bind-api:jar:4.0.4:test
9 [INFO] | | \- jakarta.activation:jakarta.activation-api:jar:2.1.4:test
10 [INFO] | +- net.minidev:json-smart:jar:2.6.0:test
11 [INFO] | | \- net.minidev:accessors-smart:jar:2.6.0:test
12 [INFO] | | \- org.ow2.asm:asm:jar:9.7.1:test
13 [INFO] | +- org.assertj:assertj-core:jar:3.27.6:test
14 [INFO] | | \- net.bytebuddy:byte-buddy:jar:1.17.8:test
15 [INFO] | +- org.awaitility:awaitility:jar:4.3.0:test
16 [INFO] | +- org.hamcrest:hamcrest:jar:3.0:test
17 [INFO] | +- org.junit.jupiter:junit-jupiter:jar:6.0.2:test
18 [INFO] | | +- org.junit.jupiter:junit-jupiter-api:jar:6.0.2:test
19 [INFO] | | | +- org.opentest4j:opentest4j:jar:1.3.0:test
20 [INFO] | | | +- org.junit.platform:junit-platform-commons:jar:6.0.2:test
21 [INFO] | | | \- org.apiguardian:apiguardian-api:jar:1.1.2:test
22 [INFO] | | +- org.junit.jupiter:junit-jupiter-params:jar:6.0.2:test
23 [INFO] | | \- org.junit.jupiter:junit-jupiter-engine:jar:6.0.2:test
24 [INFO] | | \- org.junit.platform:junit-platform-engine:jar:6.0.2:test
25 [INFO] | +- org.mockito:mockito-core:jar:5.5.0:test
26 [INFO] | | +- net.bytebuddy:byte-buddy-agent:jar:1.17.8:test
27 [INFO] | | \- org.objenesis:objenesis:jar:3.3:test
28 [INFO] | +- org.mockito:mockito-junit-jupiter:jar:5.5.0:test
29 [INFO] | +- org.skyscreamer:jsonassert:jar:1.5.3:test
30 [INFO] | | \- com.vaadin.external.google:android-json:jar:0.0.20131108.vaadin1:test
31 [INFO] | +- org.springframework:spring-core:jar:7.0.3:compile
32 [INFO] | | +- commons-logging:commons-logging:jar:1.3.5:compile
33 [INFO] | | \- org.jspecify:jspecify:jar:1.0.0:compile
34 [INFO] | +- org.springframework:spring-test:jar:7.0.3:test
35 [INFO] | \- org.xmlunit:xmlunit-core:jar:2.10.4:test
```



JUnit: The Testing Foundation

- Java's de-facto standard testing framework - version 6 with Spring Boot 4 (drop-in upgrade, unlike 4 → 5)
- More than just `@Test` : **Extension API** (replaces `@RunWith`), lifecycle hooks, `@ParameterizedTest` , `@Nested` , `@DisplayName` , **Parallel execution**

```
1 @ExtendWith(MockitoExtension.class)
2 class DiscountCalculatorTest {
3
4     @ParameterizedTest
5     @CsvSource({ "0, 0.0", "100, 10.0", "1000, 150.0" })
6     void shouldApplyDiscountWhenAmountIsValid(int amount, double expected) {
7         // ...
8     }
9 }
```



Mockito: Stub, Verify, and Beyond

- **Stubbing** with `when(...).thenReturn(...)` and **verifying** interactions with `verify(...)`
- **Argument matchers** (`any()`, `eq()`, `argThat(...)`) for flexible expectations
- **Advanced**: deep stubs (`RETURNS_DEEP_STUBS`) and static mocking (`MockedStatic`) for legacy/utility code

```
1 when(customerRepository.findById(42L))  
2   .thenReturn(Optional.of(customer));  
3  
4 verify(eventPublisher).publish(any(CustomerCreated.class));
```



AssertJ & Hamcrest: Readable Assertions

- **AssertJ**: fluent, chainable, IDE-friendly auto-completion
- **Hamcrest**: composable matchers, useful with matcher-driven APIs like Mockito
`argThat(...)`
- Pick one for general assertions and stick with it within a test class

```
1 // AssertJ – chainable & expressive
2 assertThat(customers)
3   .hasSize(3)
4   .extracting(Customer::firstName)
5   .containsExactly("Alice", "Bob", "Charlie");
6
7 // Hamcrest – composable matchers
8 assertThat("duke".toUpperCase(), equalTo("DUKE"));
```



JsonPath & JSONAssert: Working With JSON

- **JsonPath**: query JSON like XPath - drill into responses without deserializing
- **JSONAssert**: compare JSON strings with **lenient** or **strict** mode - order-insensitive, ignores extra fields

```
1 String json = "{ ... }";
2
3 // JsonPath - query a JSON document directly
4 String firstName = JsonPath.parse(json).read("$.firstName", String.class);
5 Long tagCount = JsonPath.parse(json).read("$.tags.length()", Long.class);
6
7 // JSONAssert - lenient: extra fields in actual are OK
8 String expected = "{ \"name\": \"duke\" }";
9 String actual = "{ \"name\": \"duke\", \"age\": 42 }";
10
11 JSONAssert.assertEquals(expected, actual, JSONCompareMode.LENIENT);
```



XMLUnit: Comparing XML Documents

- Compare and validate XML with **whitespace-aware, namespace-aware, order-tolerant** diffs
- Still relevant for SOAP, configuration files, and legacy enterprise integrations

```
1 String control = "<customer>...</customer>";
2 String test = "<customer>...</customer>";
3
4 Diff diff = DiffBuilder.compare(control)
5     .withTest(test)
6     .ignoreWhitespace()
7     .checkForSimilar()
8     .build();
9
10 assertThat(diff.hasDifferences()).assertFalse();
```



Awaitility: Taming Asynchronous Tests

- Fluent **polling** for eventually-consistent assertions - message queues, async event handlers, scheduled jobs
- Replaces brittle `Thread.sleep(...)` with explicit conditions and timeouts

```
1 await()  
2   .atMost(5, SECONDS)  
3   .pollInterval(100, MILLISECONDS)  
4   .untilAsserted(() ->  
5     assertThat(orderRepository.findAll()).hasSize(1)  
6   );
```



Unit Testing Java/Spring Boot Applications 101

- **Core Concept:** Test individual components in isolation from dependencies - one unit of work at a time.
- **Confidence Gained:** Fast, high-volume verification that the smallest building blocks behave correctly under various conditions.
- **Pitfall:** Poor class design leads to untestable god classes. Good tests start with good design.
- **Tools:** JUnit, Mockito, AssertJ (or Spock, TestNG, Hamcrest).



Unit Testing has Limits

Consider this sample REST controller, what could we verify with a unit test?

```
1 @RestController
2 @RequestMapping("/api/customers")
3 public class CustomerController {
4
5     private final CustomerService customerService;
6
7     public CustomerController(CustomerService customerService) {
8         this.customerService = customerService;
9     }
10
11     @PostMapping
12     public ResponseEntity<Void> createNewCustomer(@Validated CustomerCreationRequest payload, UriComponentsBuilder uriBuilder) {
13
14         String customerId = customerService.createNewCustomer(payload.firstName());
15
16         UriComponents uriComponents = uriBuilder
17             .path("/api/customers/{id}")
18             .buildAndExpand(customerId);
19
20         return ResponseEntity.created(uriComponents.toUri()).build();
21     }
22 }
```



```
1 @ExtendWith(MockitoExtension.class)
2 class CustomerControllerUnitTests {
3
4     @Mock
5     private CustomerService customerService;
6
7     @InjectMocks
8     private CustomerController customerController;
9
10    @Test
11    void shouldCreateCustomerWhenPayloadRequestIsValid() {
12        when(customerService.createNewCustomer(anyString()))
13            .thenReturn("42");
14
15        ResponseEntity<Void> result = customerController.createNewCustomer(
16            new CustomerCreationRequest("Java", "Duke", "duke@spring.io"),
17            UriComponentsBuilder.newInstance()
18        );
19
20        assertThat(result.getStatusCode().value())
21            .isEqualTo(201);
22        assertThat(result.getHeaders().getLocation().toString())
23            .isEqualTo("/api/customers/42");
24    }
25 }
```



Things We Can't Cover With a Unit Test #0

Request Mapping: Does HTTP GET `/api/customers/{id}` actually resolve to our desired method?

```
1 @Test
2 void shouldCreateCustomerWhenPayloadRequestIsValid() {
3
4     // ...
5
6     ResponseEntity<Void> result = customerController.createNewCustomer(
7         new CustomerCreationRequest("Java", "Duke", "duke@spring.io"),
8         UriComponentsBuilder.newInstance()
9     );
10 }
```



Things We Can't Cover With a Unit Test #1

Validation: Will an incomplete request body result in a 400 bad request or return an accidental 201?

```
1 @Test
2 void shouldCreateCustomerWhenPayloadRequestIsValid() {
3
4     // ...
5
6     ResponseEntity<Void> result = customerController.createNewCustomer(
7         new CustomerCreationRequest("Java", "Duke", "NOT_AN_EMAIL"),
8         UriComponentsBuilder.newInstance()
9     );
10 }
```



Things We Can't Cover With a Unit Test #2

Serialization: Are we JSON objects serialized and deserialized correctly?

```
1 ResponseEntity<Void> result = customerController.createNewCustomer(  
2     new CustomerCreationRequest("Java", "Duke", "NOT_AN_EMAIL"),  
3     UriComponentsBuilder.newInstance());
```

```
1 {  
2     "first-name": "Java",  
3     "last_Name": "Duke",  
4     "email": "duke@spring.io"  
5 }
```



Things We Can't Cover With a Unit Test #3

Security: Are we Spring Security configuration and other authorization checks enforced?

```
1 @Test
2 void shouldCreateCustomerWhenPayloadRequestIsValid() {
3
4     // ...
5
6     ResponseEntity<Void> result = customerController.createNewCustomer(
7         new CustomerCreationRequest("Java", "Duke", "NOT_AN_EMAIL"),
8         UriComponentsBuilder.newInstance()
9     );
10 }
```



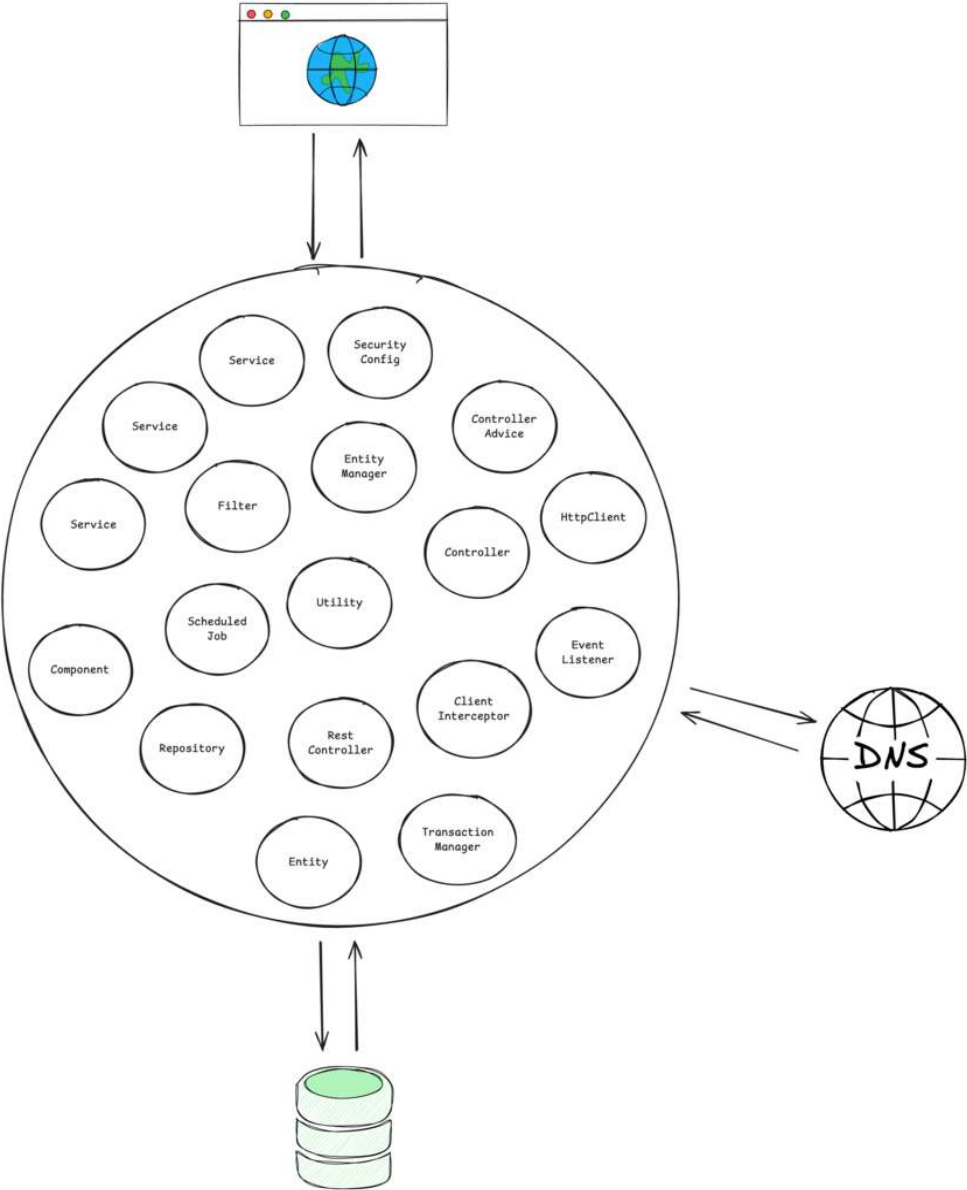
Quest 2

The Slice Testing Hydra

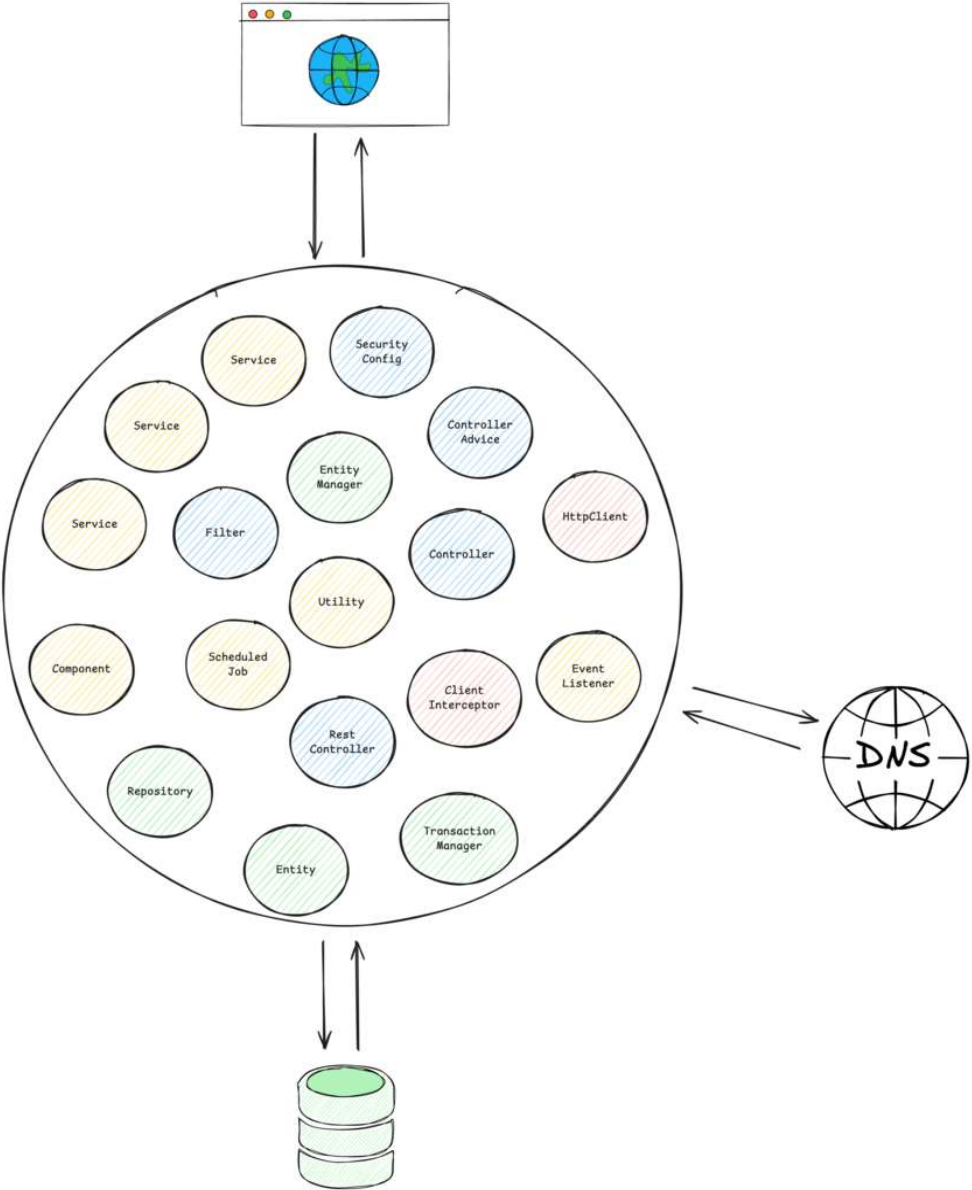
Multiple Heads, Each Guarding a Layer

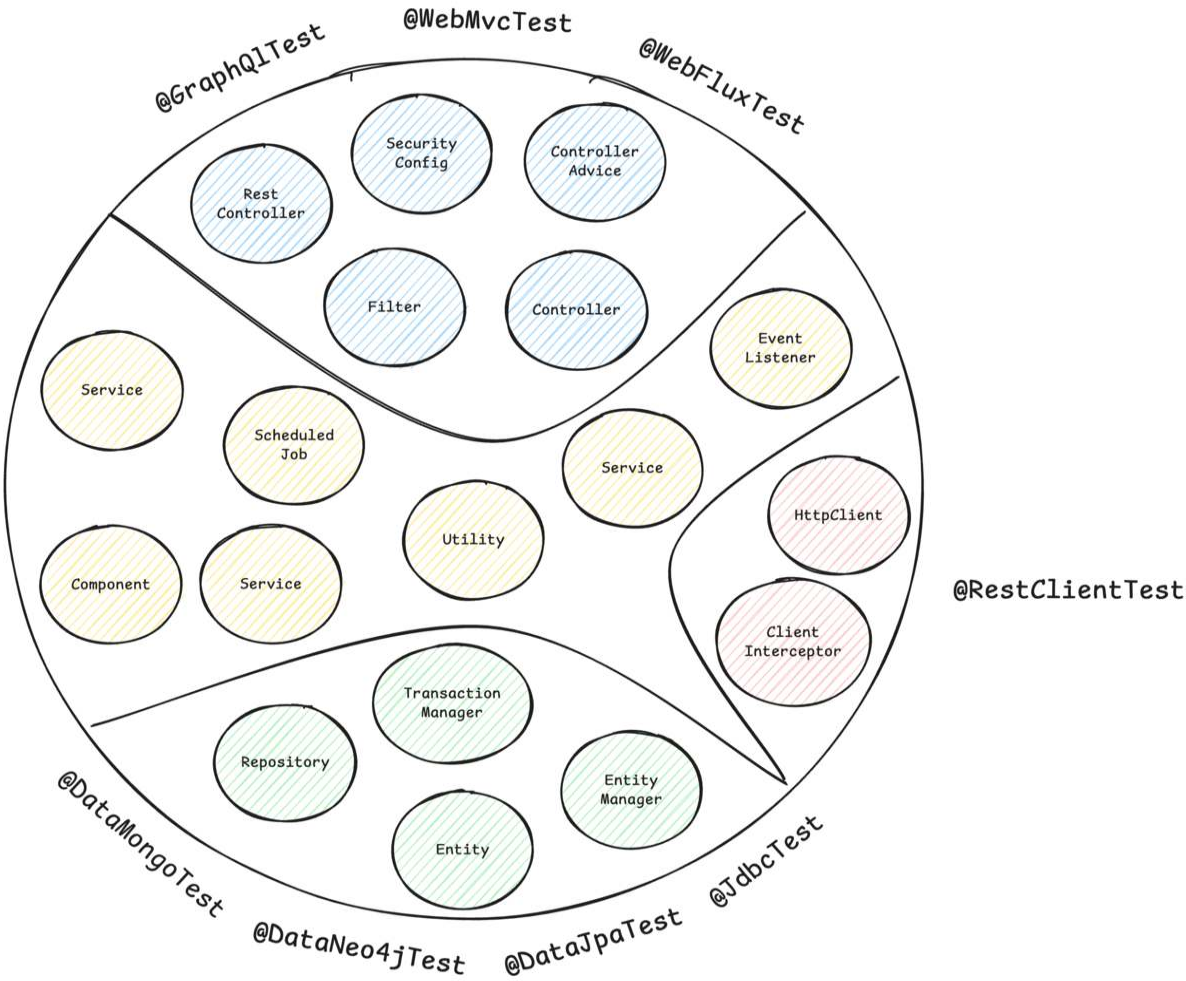


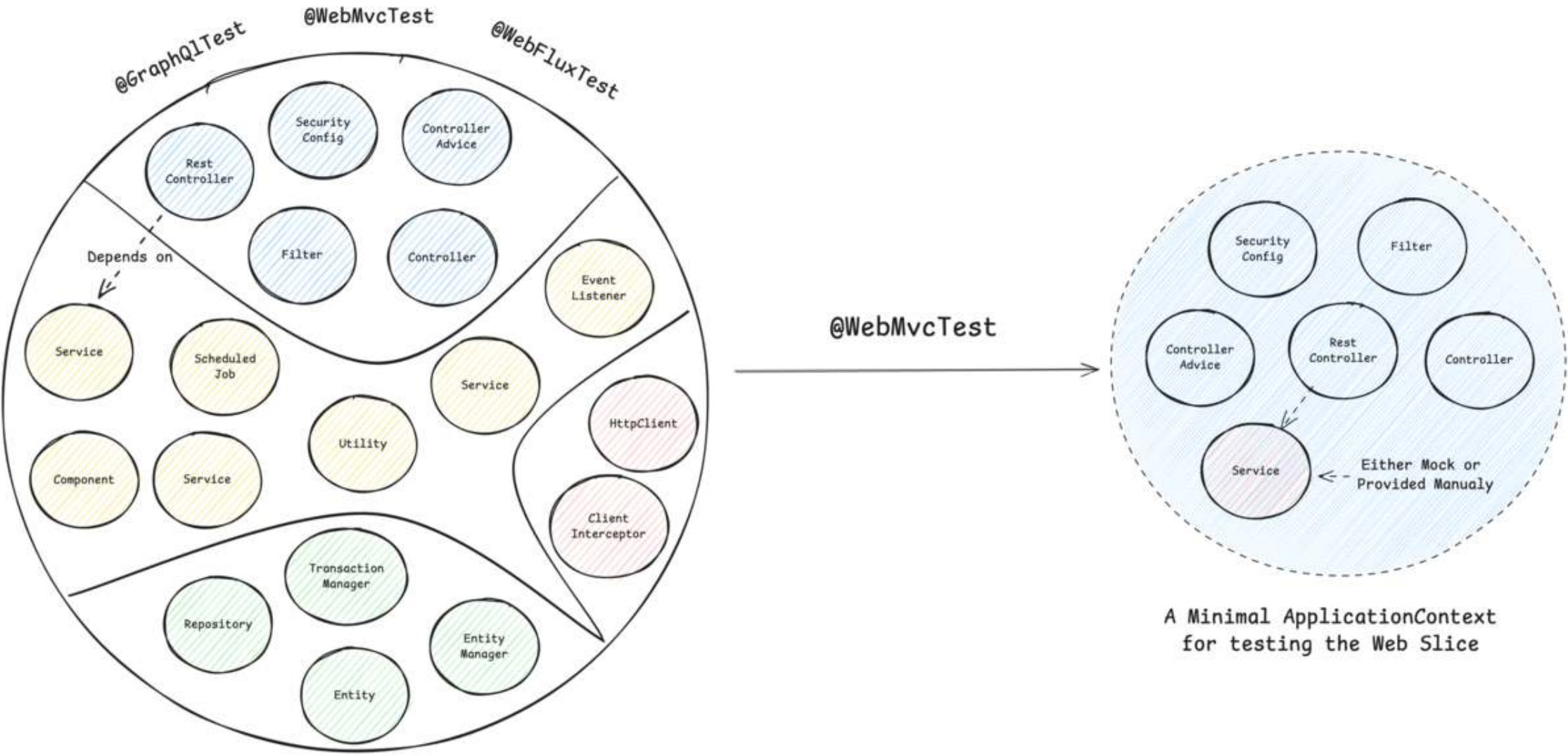
A Typical
Spring ApplicationContext
with different
Spring Beans



A Typical
Spring ApplicationContext







Spring Boot Test Slice Example: @WebMvcTest

```
1 @WebMvcTest(CustomerController.class)
2 @Import(SecurityConfig.class)
3 class CustomerControllerTest {
4
5     @Autowired
6     private MockMvc mockMvc;
7
8     @MockitoBean
9     private CustomerService customerService;
10
11     @Test
12     @WithMockUser
13     void shouldReturnLocationOfNewlyCreatedCustomer() throws Exception {
14         // ...
15     }
16 }
```



Common Test Slices

- `@WebMvcTest` / `@WebFluxTest` - Controller layer
- `@DataJpaTest` / `@JdbcTest` - Persistence layer
- `@JsonTest` - JSON serialization/deserialization
- `@RestClientTest` - RestTemplate testing
- etc.



`@DataCouchbaseTest`
`@DataLdapTest`
`@RestClientTest`
`@JsonTest`
`@JooqTest`
`@WebFluxTest`
`@DataRedisTest`
`@WebMvcTest`
`@DataMongoTest`
`@DataCassandraTest`
`@GraphQLTest`
`@DataNeo4jTest`
`@WriteYourOwn*`
`@DataJpaTest`
`@SqsTest`
`@JdbcTest`
`@DataElasticsearchTest`



Sliced Testing Spring Boot Applications 101

- **Core Concept:** Test a specific "slice" or layer of your application by loading a minimal, relevant part of the Spring `ApplicationContext`.
- **Confidence Gained:** Helps validate parts of your application where pure unit testing is insufficient, like the web, messaging, or data layer.
- **Prominent Examples:** Web layer (`@WebMvcTest`) and database layer (`@DataJpaTest`)
- **Pitfalls:** Requires careful configuration to ensure only the necessary slice of the context is loaded.
- **Tools:** JUnit, Mockito, Spring Test, Spring Boot, Testcontainers



Quest 3

The Integration Testing Dragon

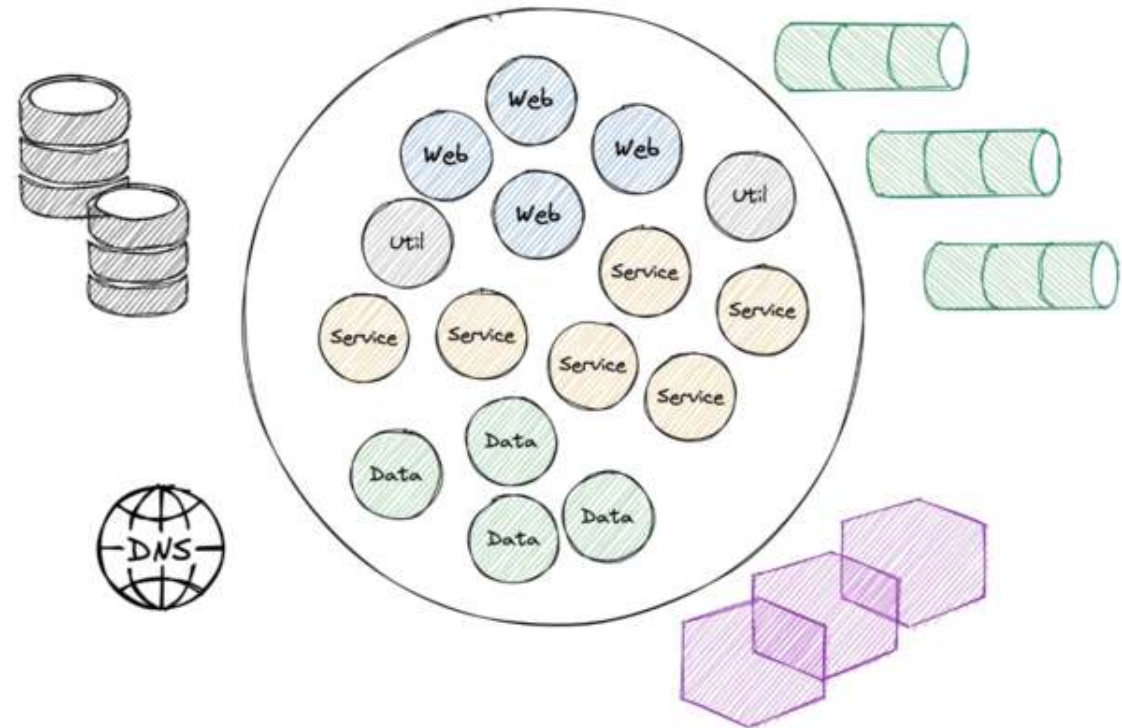
Guards the Full Treasure - but Demands Patience




```
@SpringBootTest
class ApplicationTest {

    @Test
    void contextLoads() {
    }

}
```



Challenges when Starting the Entire **ApplicationContext**

- **Problem #1:** How to Ensure Surrounding Infrastructure (e.g. database, queues, etc.) is Present?
- **Problem #2:** How to Interact with our Application for Integration Tests?
- **Problem #3:** How to Keep our Build Time at a reasonable Duration?



There's Even More...

- **Problem #4:** How to handle HTTP communication from our application to remote services?
- **Problem #5:** How to provide test data and maintain a clean state between tests?
- **Problem #6:** How to handle authentication and security contexts during tests?



Provide External Infrastructure with Testcontainers (Problem #1)

Running infrastructure components (databases, message brokers, etc.) in Docker containers for our tests becomes a breeze with [Testcontainers](#):

```
1 @Container
2 @ServiceConnection
3 static PostgreSQLContainer postgres = new PostgreSQLContainer("postgres:16-alpine")
4     .withDatabaseName("testdb")
5     .withUsername("test")
6     .withPassword("test")
7     .withInitScript("init-postgres.sql");
```

This gives us an ephemeral PostgreSQL database for our tests:

```
1 $ docker ps
2 CONTAINER ID   IMAGE                COMMAND              CREATED        STATUS        PORTS                NAMES
3 a958ee2887c6   postgres:16-alpine   "docker-entrypoint.s..." 10 seconds ago Up 9 seconds   0.0.0.0:32776->5432/tcp, [::]:32776->5432/tcp affectionate_cannon
4 ad0f804068dc   testcontainers/ryuk:0.12.0 "/bin/ryuk"         10 seconds ago Up 9 seconds   0.0.0.0:32775->8080/tcp, [::]:32775->8080/tcp testcontainers-ryuk-1f9f76a6-46d4-4e19-85c1-e8364da12804
```



How to Interact with our Application for Integration Tests?

Option #1:

```
1 @SpringBootTest
2 // which is @SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.MOCK)
3 @AutoConfigureMockMvc
4 class ApplicationMockWebIT {
5
6     // @LocalServerPort
7     // private int port; <-- this would fail the test, there is no local port occupied
8
9     @Test
10    @WithMockUser
11    void givenCustomersThenReturnListForAuthenticatedUser(@Autowired MockMvc mockMvc) throws Exception {
12        mockMvc
13            .perform(get("/api/customers")
14                .header(ACCEPT, APPLICATION_JSON))
15            .andExpect(status().is(200))
16            .andExpect(content().contentType(APPLICATION_JSON))
17            .andExpect(jsonPath("$.size()", is(1)));
18    }
19 }
```



How to Interact with our Application for Integration Tests?

Option #2:

```
1 @AutoConfigureWebTestClient // required since Spring Boot 4
2 @SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
3 class ApplicationServletContainerIT {
4
5     @Autowired
6     WebTestClient webTestClient;
7
8     @Test
9     void contextLoads() {
10         webTestClient
11             .get()
12             .uri("/api/customers")
13             .header("Authorization", "Basic " + Base64.getEncoder().encodeToString("user:dummy".getBytes()))
14             .exchange()
15             .expectStatus()
16             .isOk();
17     }
18 }
```



Integration Testing Spring Boot Applications 101

- **Core Concept:** Start the entire Spring application context, often on a random local port, and test the application through its external interfaces (e.g., REST API).
- **Confidence Gained:** Validates the integration of all internal components working together as a complete application.
- **Best Practices:** Use `@SpringBootTest` to run the app on a local port.
- **Pitfalls:** Slower to run than unit or sliced tests. Managing the lifecycle of dependent services can be complex.
- **Tools:** JUnit, Mockito, Spring Test, Spring Boot, Testcontainers, WireMock (for mocking external HTTP services), Selenium (for browser-based UI testing)



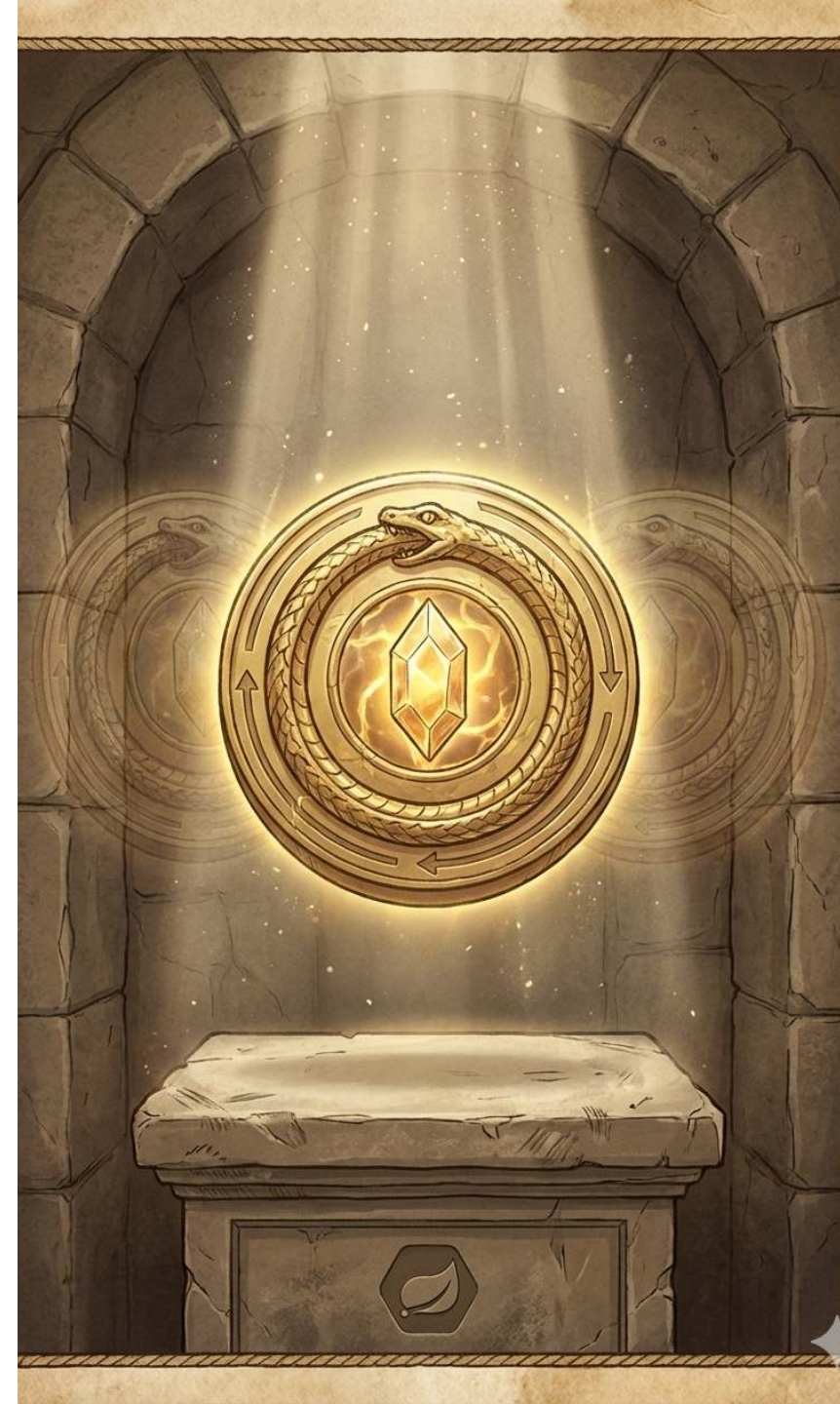
... but what about **Problem #3**: How to Keep our Build Time at a reasonable Duration?



Quest Item 1

The Caching Amulet

Helps You Reuse What You Already Built



Integration Testing - The Need for Speed

- **The Problem:** Integration tests require a started & initialized Spring `ApplicationContext`, which slows down the build
- **The Solution:** Spring Test `TestContext` Caching – stores an already started Spring `ApplicationContext` for reuse
- This feature is part of Spring Test (included in every Spring Boot project via `spring-boot-starter-test`)

Example of speed improvement:



Test run time: 3506ms

OrderIT
requiring context
configuration with the
hash code #327321289

PaymentIT
requiring context
configuration with the
hash code #327321289

CheckoutIT
requiring context
configuration with the
hash code #565212314

Context cache lookup

Spring TestContext Context Cache

Cache Key

Cache Value

no cached context,
tests needs
to start the context



Test run time: 3506ms

OrderIT
requiring context
configuration with the
hash code #327321289

Test run time: 406ms

PaymentIT
requiring context
configuration with the
hash code #327321289

CheckoutIT
requiring context
configuration with the
hash code #565212314

test gets cached context and
finishes the test very fast

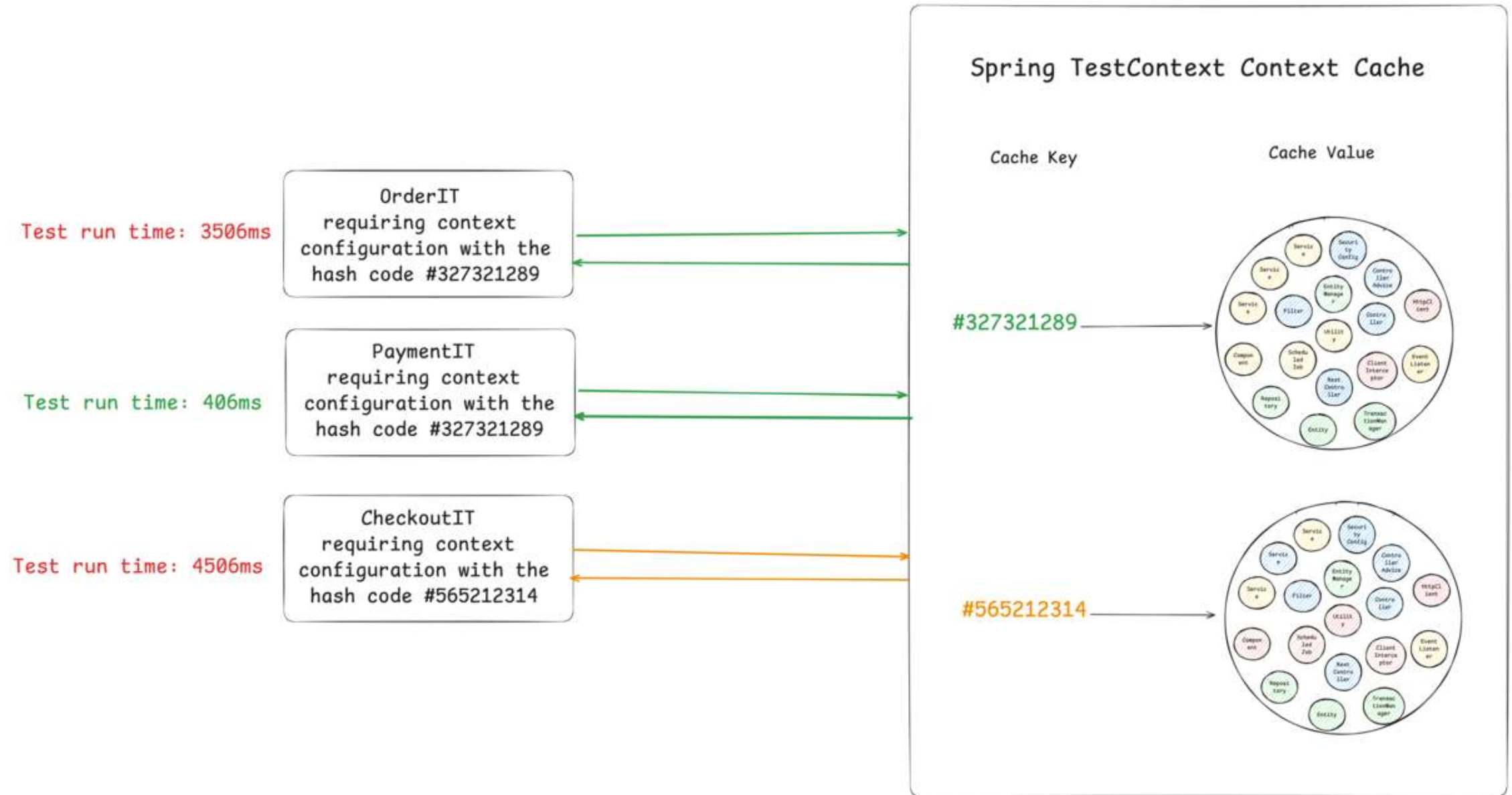
Spring TestContext Context Cache

Cache Key

Cache Value

#327321289





How the Cache Key is Built

```
1 // DefaultContextCache.java
2 private final Map<MergedContextConfiguration, ApplicationContext> contextMap =
3     Collections.synchronizedMap(new LinkedHashMap<>(32, 0.75f, true));
```

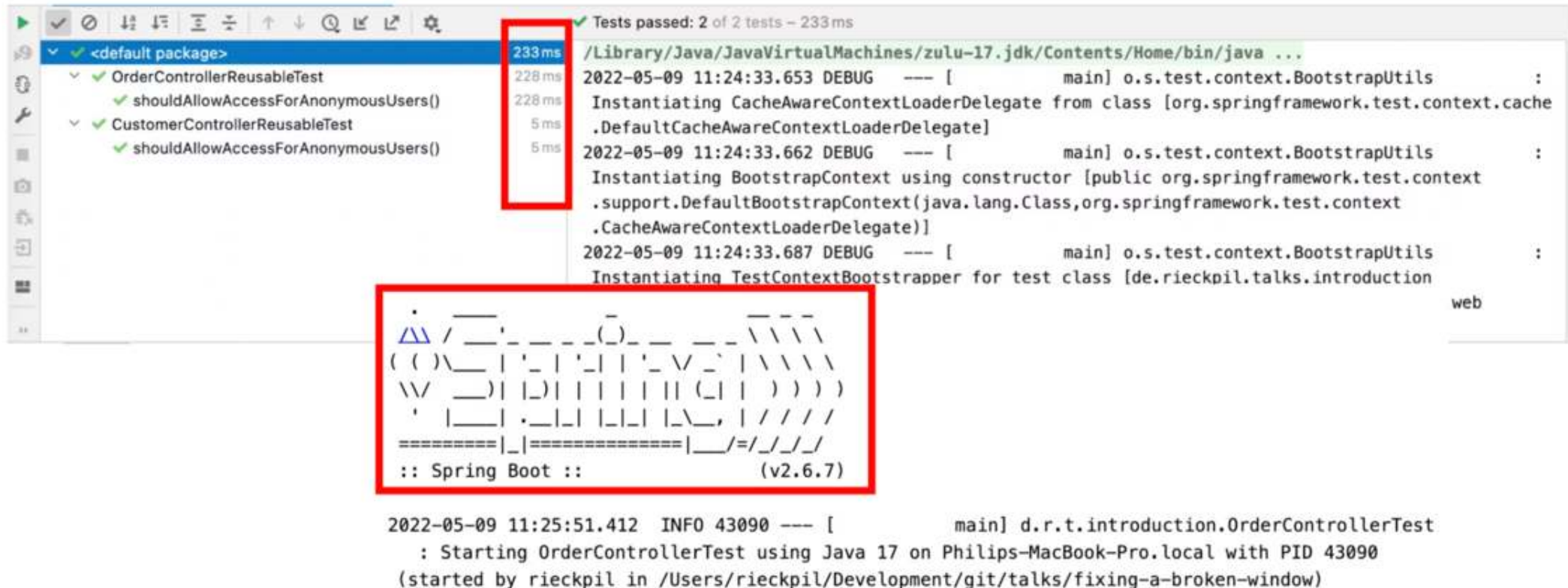
The following information is part of the Cache Key (`MergedContextConfiguration`):

- `activeProfiles` (`@ActiveProfiles`)
- `contextInitializersClasses` (`@ContextConfiguration`)
- `propertySourceLocations` (`@TestPropertySource`)
- `propertySourceProperties` (`@TestPropertySource`)
- `contextCustomizer` (`@MockitoBean` , `@MockBean` , `@DynamicPropertySource` , ...)
- etc.



```
1 Test class
2   |
3   ▼
4 MergedContextConfiguration(
5   testClass, locations, classes,
6   activeProfiles, propertyValues,
7   contextInitializers, contextCustomizers ← every @MockitoBean lands here
8   ... etc.
9 )
10  |
11  ▼ hashCode() / equals()
12
13 Cache hit? → reuse context ✓
14 Cache miss? → start new context and store it NEW
```





Detect Context Restarts - with Logs

```
<logger name="org.springframework.test.context" level="TRACE" />
```

```
2022-05-05 14:27:38.136 DEBUG 42500 --- [          main] org.springframework.test.context.cache : Spring test
ApplicationContext cache statistics: [DefaultContextCache@68e62b3b size = 1, maxSize = 32, parentContextCount = 0,
hitCount = 11, missCount = 1]
2022-05-05 14:27:38.136 DEBUG 42500 --- [          main] tractDirtiesContextTestExecutionListener : After test class:
context [DefaultTestContext@325bb9a6 testClass = ApplicationIT, testInstance = [null], testMethod = [null], testException
= [null], mergedContextConfiguration = [WebMergedContextConfiguration@1d12b024 testClass = ApplicationIT, locations =
'{}', classes = '{class de.rieckpil.talks.Application}', contextInitializerClasses = '[]', activeProfiles = '{}',
propertySourceLocations = '{}', propertySourceProperties = '{org.springframework.boot.test.context
.SpringBootTestContextBootstrapper=true, server.port=0}', contextCustomizers = set[org.springframework.boot.test.context
.filter.ExcludeFilterContextCustomizer@5215cd9a, org.springframework.boot.test.json
.DuplicateJsonObjectContextCustomizerFactory$DuplicateJsonObjectContextCustomizer@9257031, org.springframework.boot.test
```



Detect Context Restarts - with Tooling



An [open-source Spring Test utility](#) that provides visualization and insights for Spring Test execution, with a focus on Spring context caching statistics.

Overall goal: Identify optimization opportunities in your Spring Test suite to speed up your builds and ship to production faster and with more confidence.



The Final Boss

Developers tend to consult AI/StackOverflow for integration test issues and often copy advice from the internet without knowing the implications:

```
1 @SpringBootTest
2 @DirtiesContext
3 // this instructs Spring to remove the context from the cache
4 // and rebuild a new context on every request
5 public abstract class AbstractIntegrationTest {
6
7 }
```

The setup above will **disable** the context caching feature and slow down the builds significantly!



New in Spring Framework 7: Pausing Contexts

See Release Notes von [Spring Framework 7](#).

Pausing of Test Application Contexts

The Spring TestContext framework is caching application context instances within test suites for faster runs. As of Spring Framework 7.0, we now pause test application contexts when they're not used.

This means an application context stored in the context cache will be stopped when it is no longer actively in use and automatically restarted the next time the context is retrieved from the cache.

Specifically, the latter will restart all auto-startup beans in the application context, effectively restoring the lifecycle state.



Quest Item 2

The Lightning Shield

Many Cores, One Goal



Test Parallelization

Goal: Reduce build time and get faster feedback

Requirements:

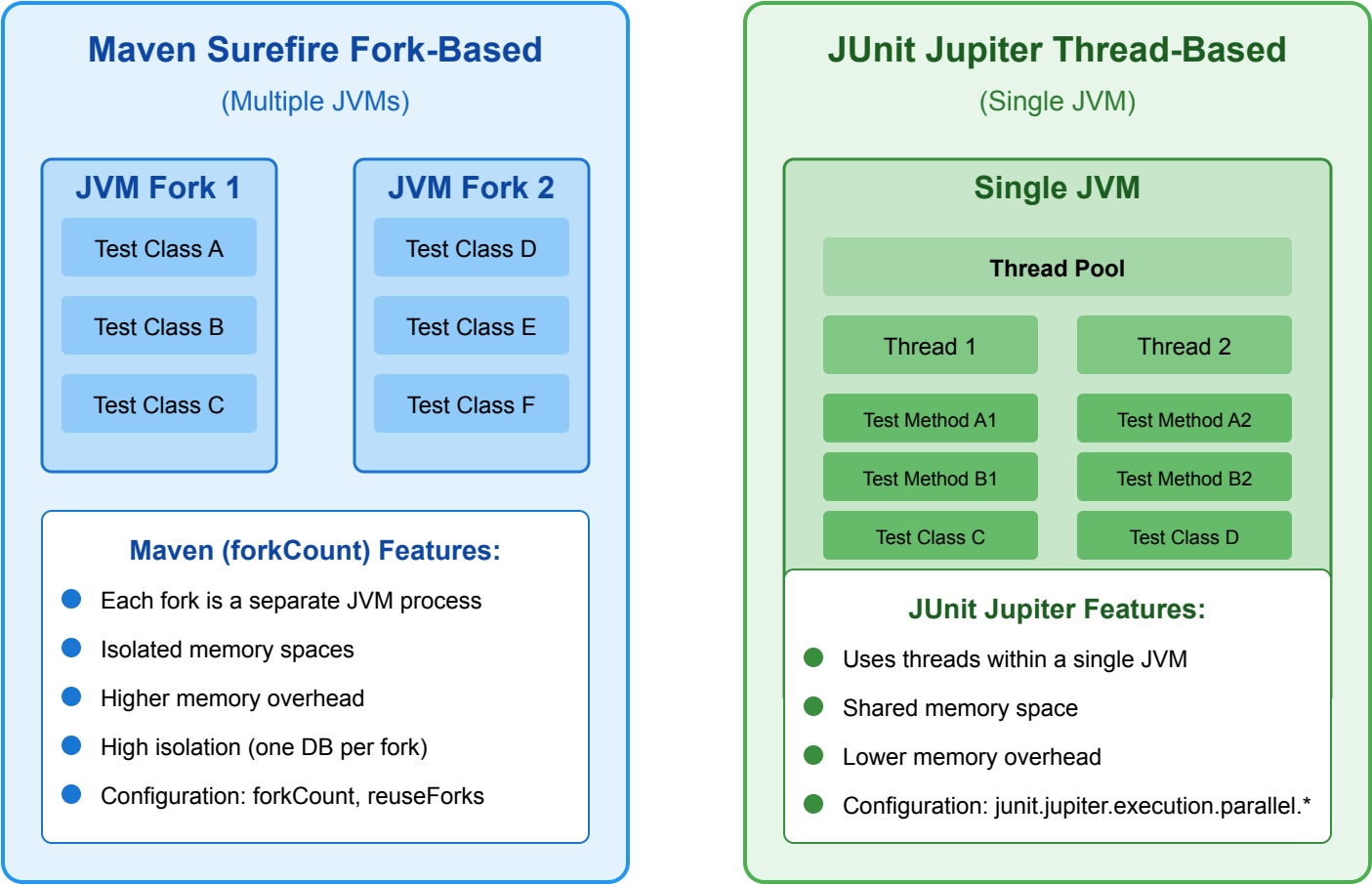
- No shared state
- No dependency between tests and their execution order
- No mutation of global state

Two ways to achieve this:

- Fork a new JVM with Surefire/Failsafe (or for the Gradle test task) and let it run in parallel
- Use JUnit Jupiter's parallelization mode and let it run in the same JVM with multiple threads



Java Test Parallelization Options



Test Parallelization 101

Using Surefire/Failsafe:

```
1 <plugin>
2   <artifactId>maven-surefire-plugin</artifactId>
3   <configuration>
4     <forkCount>1C</forkCount> <!-- 1 JVM per CPU core -->
5   </configuration>
6 </plugin>
```

With JUnit Jupiter:

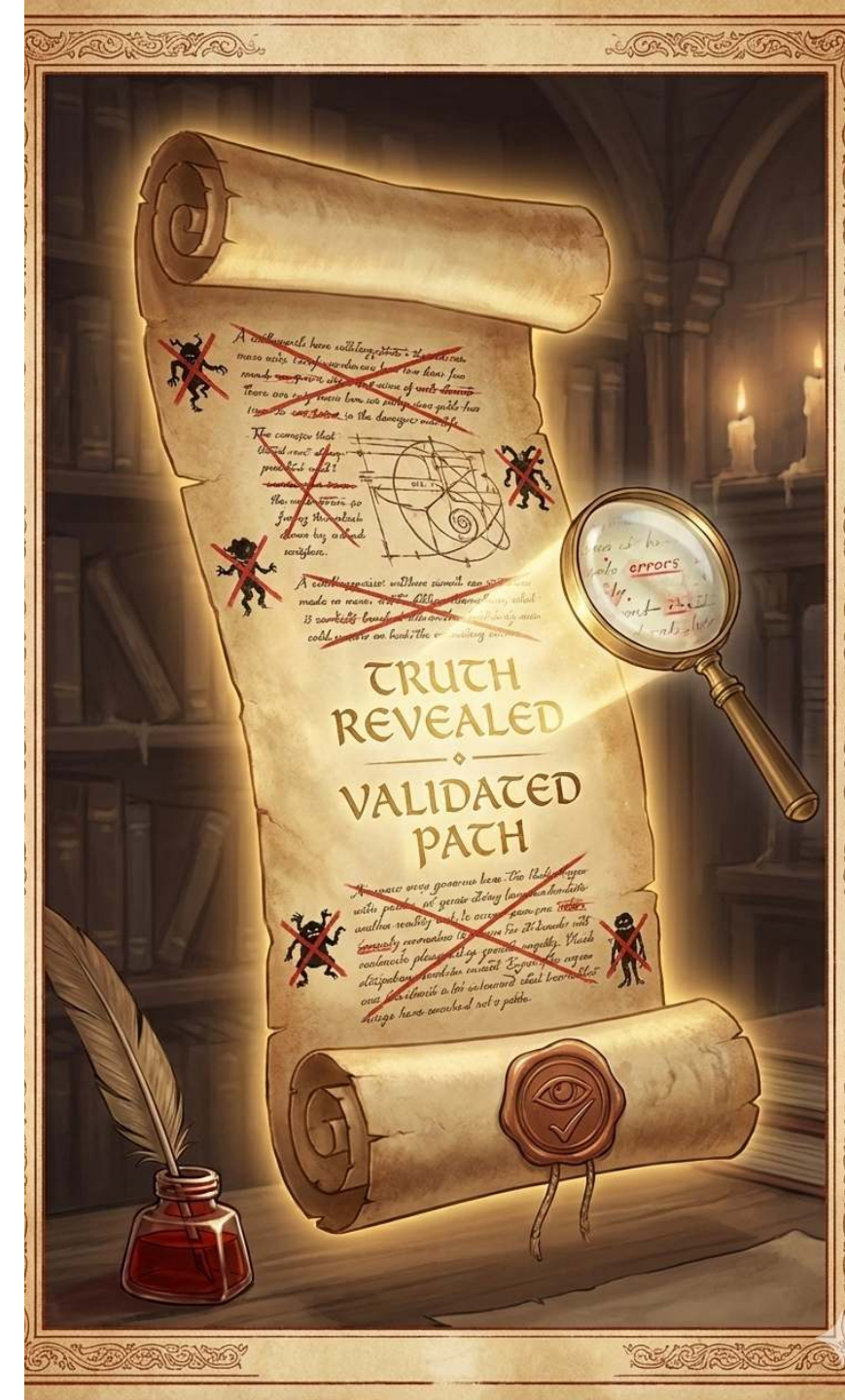
```
1 # src/test/resources/junit-platform.properties
2 junit.jupiter.execution.parallel.enabled = true
3 junit.jupiter.execution.parallel.mode.default = same_thread
4 junit.jupiter.execution.parallel.mode.classes.default = concurrent
```



Quest Item 3

The Scroll of Truth

Coverage Lies, Mutants Don't

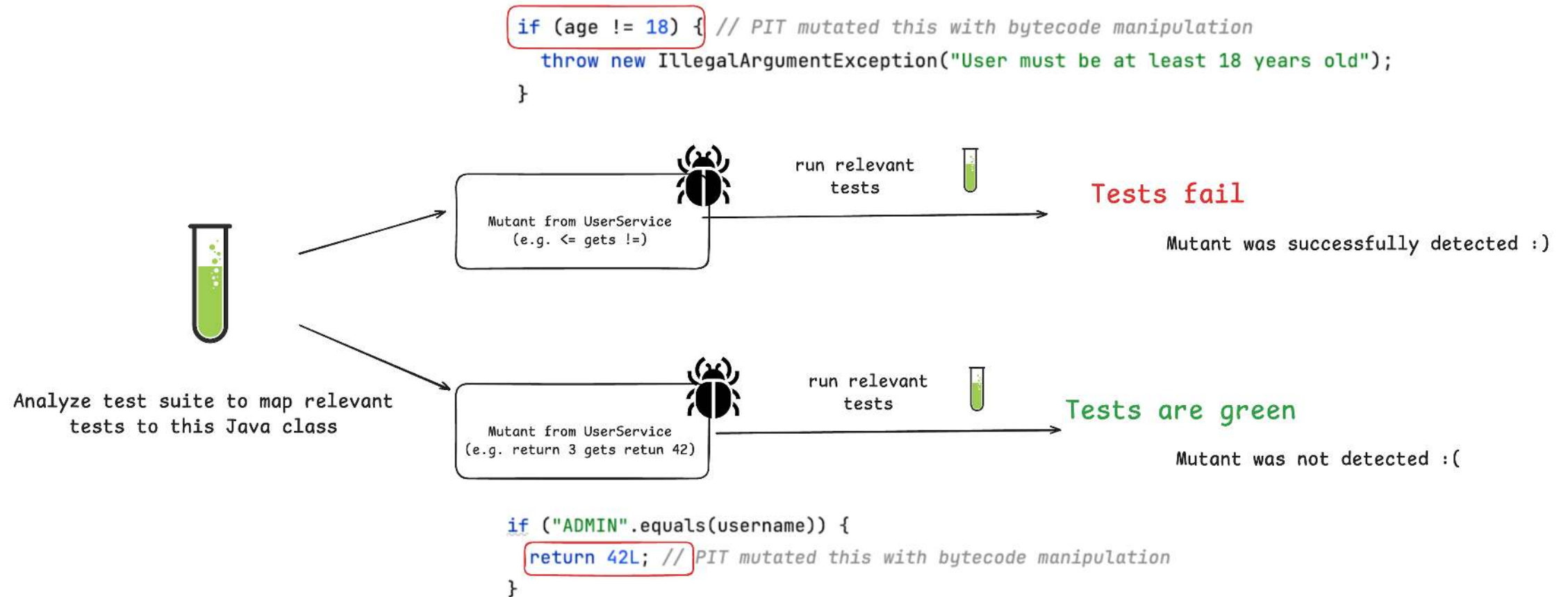


Let's Challenge Code Coverage

```
1 public Long registerUser(int age, String username) {  
2  
3     if (age <= 18) {  
4         throw new IllegalArgumentException("User must be at least 18 years old");  
5     }  
6  
7     if ("ADMIN".equalsIgnoreCase(username)) {  
8         throw new IllegalArgumentException("Username 'ADMIN' is not allowed");  
9     }  
10  
11     // ...  
12  
13 }
```



Idea: Introduce Regressions to Verify Test Quality



Introducing: Mutation Testing

- Having high code coverage might give you a **false sense of security**
- Mutation Testing with **PIT**
- Beyond Line Coverage: Traditional tools like JaCoCo show which code runs during tests, but PIT verifies if our tests actually detect when code behaves incorrectly by introducing "**mutations**" to our source code.
- Quality Guarantee: PIT automatically **modifies our code** (changing conditionals, return values, etc.) to ensure our tests fail when they should, **revealing blind spots** in seemingly comprehensive test suites.



Act 5: The Triumphant Exit

- Spring Boot applications come with batteries-included and excellent testing support
- We've completed three main quests: Unit testing, Sliced testing, and Integration testing
- Three core quest items help us to speed up and validate our tests:
 - Context Caching
 - Test Parallelization
 - Mutation Testing



Spring Boot 4 - Testing Support Keeps Improving

- **RestTestClient:** Modern, fluent alternative for the `TestRestTemplate` / `WebTestClient` / `RestAssured`.
- **Context pausing:** Cached test contexts are now automatically paused, eliminating resource conflicts from background processes.
- **JUnit 6:** Drop-in upgrade from JUnit 5 - far smoother than the JUnit 4 → 5 migration.
- **Testcontainers 2.0:** New `testcontainers-` prefix for modules, JUnit 4 support removed.
- **Bean overrides for non-singletons:** `@MockitoBean` and `@TestBean` now work with prototype and custom-scoped beans.



Making the Most of Agentic Development

- Invest in a **comprehensive** and **fast test suite**
- Reduce context switches for developers to stay in the loop
- Achieve **operational excellence**: monitoring, alerting, runbooks, A/B testing, feature flags, canary testing etc.
- Use e.g. Playwright MCP to let the AI self-verify features locally
- Provide custom skills to standardize code and tests across the company



Optimizing AI-written Tests with Skills

spring-boot-testing/	
— SKILL.md	# router: picks the cheapest test level, links to subskills
— unit-testing/	
— SKILL.md	# JUnit 5 + Mockito, no Spring context
— examples.md	
— slice-testing/	
— SKILL.md	# @DataJpaTest, @RestClientTest, @JsonTest, MockWebServer
— examples.md	
— reference.md	# annotation surface + Boot 4 import paths
— slice-testing-webmvc/	
— SKILL.md	# @WebMvcTest + MockMvc + security
— examples.md	
— integration-testing/	
— SKILL.md	# @SpringBootTest + shared base class
— examples.md	
— testcontainers-setup/	
— SKILL.md	# @ServiceConnection vs DynamicPropertySource, singletons/reuse
— examples.md	
— reference.md	# @ServiceConnection support matrix, lifecycle cheat-sheet
— test-setup-reviewer/	
— SKILL.md	# audit workflow (context-cache + parallel safety)
— reference.md	# anti-pattern catalogue + JUnit parallel config
— scripts/	
— audit_test_setup.py	# working scanner (tested on this repo)



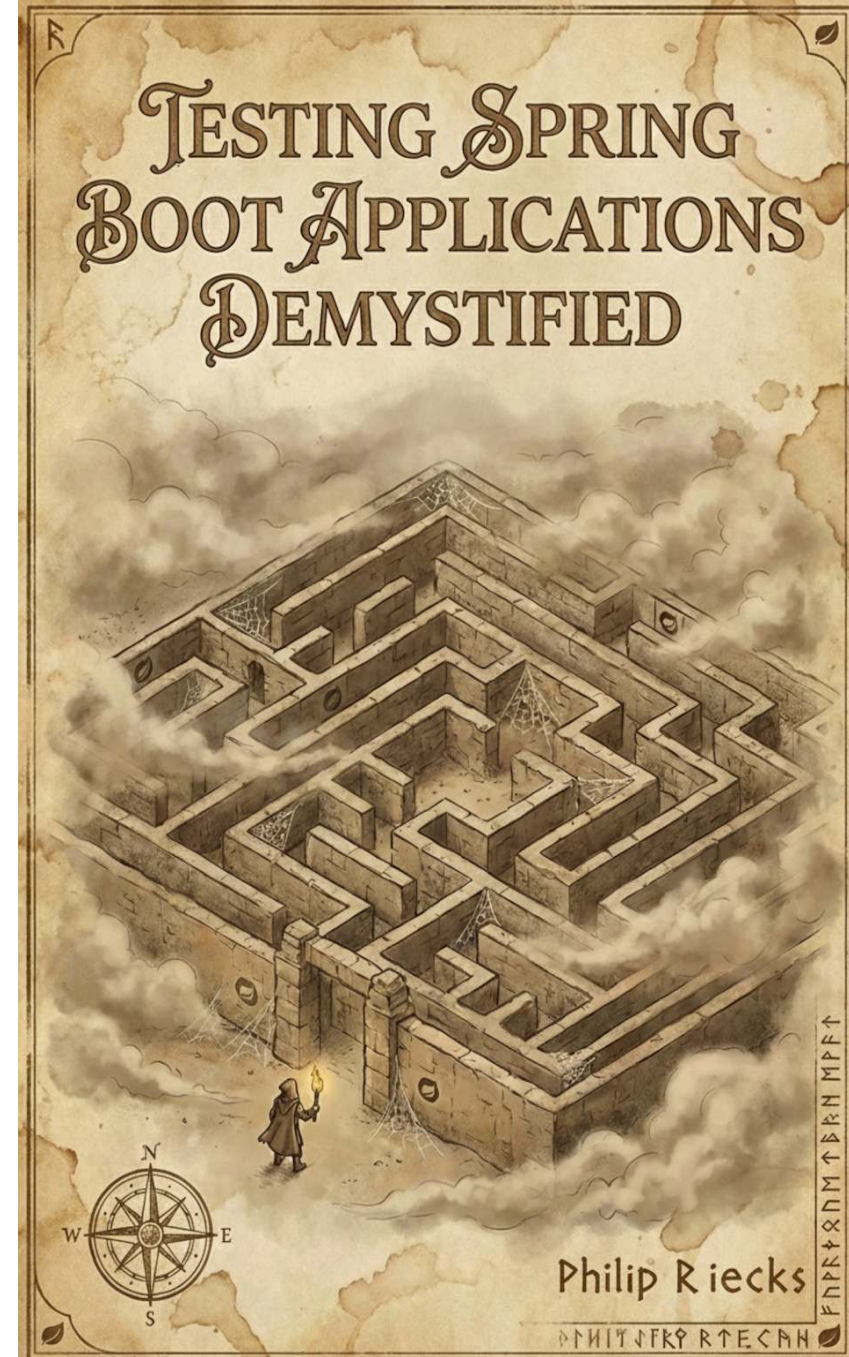
Let's Finalize the Menti Questions

Go to menti.com and use the code **7495 6357** to **anonymously** submit answers for the quizzes and add your questions during the talk.



Don't Leave Empty-Handed

- Get the complementary **Testing Spring Boot Applications Demystified** for free (instead of \$9)
- 120+ Pages with practical hands-on advice to ship code with confidence
- Get the eBook by joining our [newsletter](#)



Joyful Testing!

Get your Spring Boot Testing eBook (120+ pages):



Reach out any time via: [LinkedIn](#) (Philip Riecks) or [Mail](mailto:philip@pragmatech.digital) (philip@pragmatech.digital)

